PROJECT BRIEF — BUILD CONTEXT FOR ANTIGRAVITY

Sales Call Prep-Sheet Generator

An AI-powered B2B pre-call intelligence tool — product overview, tech stack, and build roadmap

FRONTEND	SCRAPING	LLM ACCESS	ARCHITECTURE	TEAM	TIMELINE
Streamlit	**Jina Reader**	**OpenRouter API**	**4 chained agents**	**2 people**	**No fixed deadline**

1. What We're Building

A B2B sales rep typically loses 20–30 minutes per call digging through a target company's website, the prospect's LinkedIn, and recent news to find usable talking points — nearly 2 hours a day across 5 calls. This tool automates that into a single-input workflow: the user enters a **company name**, the **prospect's job title**, and **what they're selling**. The app scrapes open company intelligence, runs it through a chain of LLM agents, and returns a clean, one-page "cheat sheet" the rep can skim in under a minute before the call.

Core output (the 4 things the sheet must contain)

Component	What it is
Golden Hook	One hyper-personalized opening line connecting the seller's product to the prospect's situation — copy-pasteable into an email or said aloud as a call opener.
Strategic Pain Points	Exactly 3 specific operational challenges that exact role is likely facing right now, grounded in the company's real context.
Icebreakers & Milestones	Recent company news (funding, launches) plus 2–3 open-ended icebreaker questions tailored to the role.
Tailored Pitch	A short paragraph explicitly mapping the seller's product to at least one identified pain point.

Why the job title matters as much as the company

The same company needs a completely different pitch depending on who's in the room — this is the product's core value, not a side feature.

Buyer A — VP of Customer Support

Cares about: agent burnout, weekend coverage gaps.

> *"Your support team is getting crushed by repetitive weekend queries. Our chatbot handles 80% of those FAQs automatically."*

Buyer B — CFO

Cares about: headcount cost, churn from slow response times.

> *"Scaling a manual support team is expensive. Our AI chatbot cuts service costs by 40% while keeping response times under 2 seconds."*

2. Tech Stack & Architecture

Layer	Tool	Role
Frontend	**Streamlit**	Input form (company, job title, product) + rendered output panel
Data fetching	**Jina Reader**	Converts company homepage/search results into clean markdown text for the LLM — no custom scraper needed
LLM access	**OpenRouter API**	Single API key (in `.env`) giving access to multiple different models, one per agent stage
Secrets	**.env**	Stores `OPENROUTER_API_KEY` and one model-name variable per agent — lets either teammate swap models without touching code

Multi-agent pipeline

Rather than one giant LLM call, the system is a chain of small, single-purpose async functions — each calls one OpenRouter model and returns one well-defined chunk of the final output. This keeps prompts short and each model's job easy to verify.

0. Scrape (Jina Reader) → **1. Company Brief** (fast/cheap model) → **2. Pain Points** (strong reasoning model) → **3. Icebreakers** (mid-tier model) → **4. Hook/Pitch** (strongest model)

Agent	Consumes	Produces
1. Company Brief	raw scraped text	`short_summary`, `recent_milestones[]`
2. Pain Points	raw text + job title	`strategic_pain_points[]` (exactly 3)
3. Icebreakers	summary + milestones + job title	`icebreaker_questions[]`
4. Hook/Pitch	ALL prior outputs + seller's product	`golden_hook`, `tailored_pitch`

Why this order: Agent 4 runs last and gets everything the earlier agents already extracted — it's the "closer" that synthesizes summary + pain points + milestones into one punchy line and a short pitch, so it never has to re-derive context.

Final merged JSON contract

All 4 agent outputs are merged into one object before reaching the Streamlit UI — the frontend never needs to know it came from 4 separate calls:

Field	Type
`short_summary`, `golden_hook`, `tailored_pitch`	string
`recent_milestones`, `icebreaker_questions`	array of strings

`strategic_pain_points`	array of 3 objects: `{challenge, why_it_matters}`
`meta.data_source`	"live" or "cached" — honest flag for fallback data

3. Build Roadmap (Phase-Based, No Fixed Deadline)

Since there's no hackathon clock, the roadmap is organized by phase/milestone rather than hour blocks — work through phases in order, at whatever pace fits a 2-person team.

Phase 1 — Skeleton
Streamlit UI: 3 inputs (company, job title, product) + generate button + empty output container. No AI logic yet.

Phase 2 — Data Layer
Jina Reader integration + 2–3 saved mock company text files as a fallback if live scraping fails or is slow.

Phase 3 — Agent Chain
Build the 4 agents one at a time (Brief → Pain Points → Icebreakers → Hook/Pitch), each as its own OpenRouter-backed async function with its own model in `.env`.

Phase 4 — Polish
Render merged JSON in Streamlit with styled cards/expanders, highlight the golden hook, add a "cached vs live" badge, optional PDF export.

Build checklist

- ☐ Streamlit skeleton: 3 inputs + generate button
- ☐ Jina Reader wrapper function
- ☐ Mock data fallback (2–3 saved company text files)
- ☐ .env with OPENROUTER_API_KEY + per-agent model vars
- ☐ Agent 1 — Company Brief function + schema validation
- ☐ Agent 2 — Pain Points function (enforce exactly 3 items)
- ☐ Agent 3 — Icebreakers function
- ☐ Agent 4 — Hook/Pitch function (consumes all prior outputs)
- ☐ Orchestrator that chains all 4 agents async
- ☐ Merge function combining outputs into final JSON
- ☐ Streamlit rendering: hook box, pain point cards, icebreaker list
- ☐ data_source badge (live vs cached)

Reliability note for the build agent: Validate each agent's JSON output against its expected schema immediately after the call, before passing it into the next agent's prompt. With 4 different OpenRouter models in the chain, don't assume all of them respect "JSON only" instructions equally well — a malformed Agent 1 output will silently corrupt Agents 3 and 4 downstream if unchecked.